

POCKET SMART AI — Your Smart Budget & Recommendation Assistant

Phase 3: Project Design

St. George's Arts and Science College (UNM1727) · TN Skills | Generative AI · Team ID: SWTID-2026-2087

Team Leader: Monish B | Members: Jagadeeshwaran S, Shannon Cadric Jacob, Preethika B, Naresh S

3.1 Purpose of This Phase

This phase turns the approved requirements into a concrete system design: the architecture layers, the technology-stack rationale, the end-to-end workflow, and the folder/module structure the implementation follows.

3.2 System Design Decisions

The system uses a modular flow that separates presentation, input handling, business logic, and data/state concerns. User-facing output is kept simple enough for a non-technical consumer while the internal logic remains maintainable for the development team.

3.3 Technology Stack Selection Justification

A browser-based architecture was selected because the problem is user-interaction heavy and benefits from easy access across devices without installation. The implementation uses AWS-hosted infrastructure (CloudFront, API Gateway, Lambda, DynamoDB, S3, Secrets Manager, IAM, CloudWatch) together with Google Gemini 1.5 Flash Pro as the generative-AI recommendation engine, as shown in the architecture diagram supplied with the project.

3.4 Technical Architecture Overview

Layer	Responsibility
Frontend Layer	User-facing pages/forms for registration, login, input collection, navigation, and dashboard presentation.
Backend / Application Layer	Validates requests, applies budgeting and recommendation logic, coordinates operations, prepares output.
Database Layer	Stores user/application data (plans, budgets, history) when persistent storage is enabled.
External AI / API Layer	Generative AI reasoning (Google Gemini) for allocation percentages and item-level recommendations.
Deployment Layer	Runs the application as a browser-accessible web app, separated into local and hosted production stages.

3.5 Specific Use-Case Budgeting Flow (as designed)

- Step 1 — User Input:** the user submits a domain (Home, Party, Jewelry), a total budget, and category details (e.g. rooms, guest count, occasion).
- Step 2 — Orchestration:** the backend (Lambda) extracts and routes the data.
- Step 3 — Prompt Construction:** the budget, domain, and priorities are mapped into a structured prompt for the AI engine.
- Step 4 — AI Logic Execution:** the generative model performs pattern recognition, allocation logic, and reasoning to produce percentage splits and item recommendations.
- Step 5 — Output Generation:** the allocation, category breakdown, and recommendation text are generated and stored, then returned to the interface.

3.6 System Workflow

Step	Description
1. User Interaction	The user opens the application and navigates to the required access/input screen.
2. Input Handling	The application collects credentials and/or budgeting requirements and validates mandatory fields.
3. Processing Logic	Validated values are passed into the budget-allocation and recommendation logic.
4. Core System Execution	The system checks constraints (total budget, selected requirements) and prepares a result.
5. Output Generation	The budget/recommendation view and supporting information are generated.
6. Response Delivery	The result is returned to the browser and shown through the dashboard/result interface.

3.7 Interface Design (from supplied screens)

The designed screen flow consists of: a landing page, account creation, login, a dashboard offering three planners (Home Interior, Party, Jewelry), a per-domain input form, a results/recommendation page, and a saved-history page — matching the screenshots captured from the working application.

3.8 Planned Project Structure

templates/ (base.html, dashboard.html, home_planner.html, party_planner.html, jewelry_planner.html, login.html, register.html, result.html, history.html, history_detail.html), static/ (assets), app.py (entry point), database.py, gemini_utils.py (AI integration), tests/, requirements.txt, and .env/.env.example for configuration.

3.9 Outcome of This Phase

A layered technical architecture, a justified technology-stack choice, a documented end-to-end workflow, and a planned file/folder structure — ready to be built out in the development phase.